LAGOON: Lightweight Animation and Graphics for Open Ocean Numerics

White paper · Sep 27, 2026 · @Hans Bihs

Summary

LAGOON is a proposed open-source tool, hosted under the REEF3D organisation on GitHub, that joins interactive 3D exploration, publication-quality plotting and cinematic rendering of ocean and coastal simulations in one lightweight application. It is meant as a leaner, faster-moving alternative to ParaView, built for hydrodynamics users rather than for general HPC visualization.

Four ideas define it:

- **Task-first design:** the user works with free surfaces, gauges, forces and slices, not with generic filter pipelines.
- **One data layer, three outputs:** matplotlib for journal figures, GPU rendering on Metal, Vulkan or Direct3D 12 for interactive 3D, and a path tracer for showcase animations.
- **A lightweight native data format:** a structured, chunked Zarr layout that fits REEF3D's Cartesian and σ-grids, with VTU kept only as a fallback reader.
- **Open from the start:** REEF3D is the first and best-supported source, but readers for other codes and for lab and field data are plugins, so the community can grow beyond REEF3D users.

Motivation

Today's open-source visualization tools force a split workflow: explore in ParaView, then rebuild every figure in a separate plotting tool. LAGOON closes that gap.

ParaView is overloaded and conservative. Its main users are national labs and HPC centres, so it must protect in-situ workflows, batch scripts and two decades of saved state files. The result is slow development and an interface built around VTK filter pipelines. Routine tasks such as showing the free surface, or plotting velocity along a gauge line, take many steps.

Figures are an afterthought. ParaView exports rarely look like journal figures. Typography, colour and layout have to be redone in matplotlib or a program like DataGraph, which breaks the link between data and figure.

Nothing joins 3D exploration and 2D plotting. Free plotting tools (Veusz, LabPlot) and free 3D tools (ParaView, VisIt) are both mature, but no open tool covers the whole path from a 3D field to a finished, styled figure or animation.

VTU does not fit structured solvers. VTU is an unstructured format: it stores explicit coordinates, connectivity and a cell type for every cell. REEF3D's grids are structured, Cartesian for CFD and σ-grids for NHFLOW and FNPF, so most of that information is redundant. A run also produces thousands of separate files, one per time step per MPI rank, which are slow to load and awkward to manage.

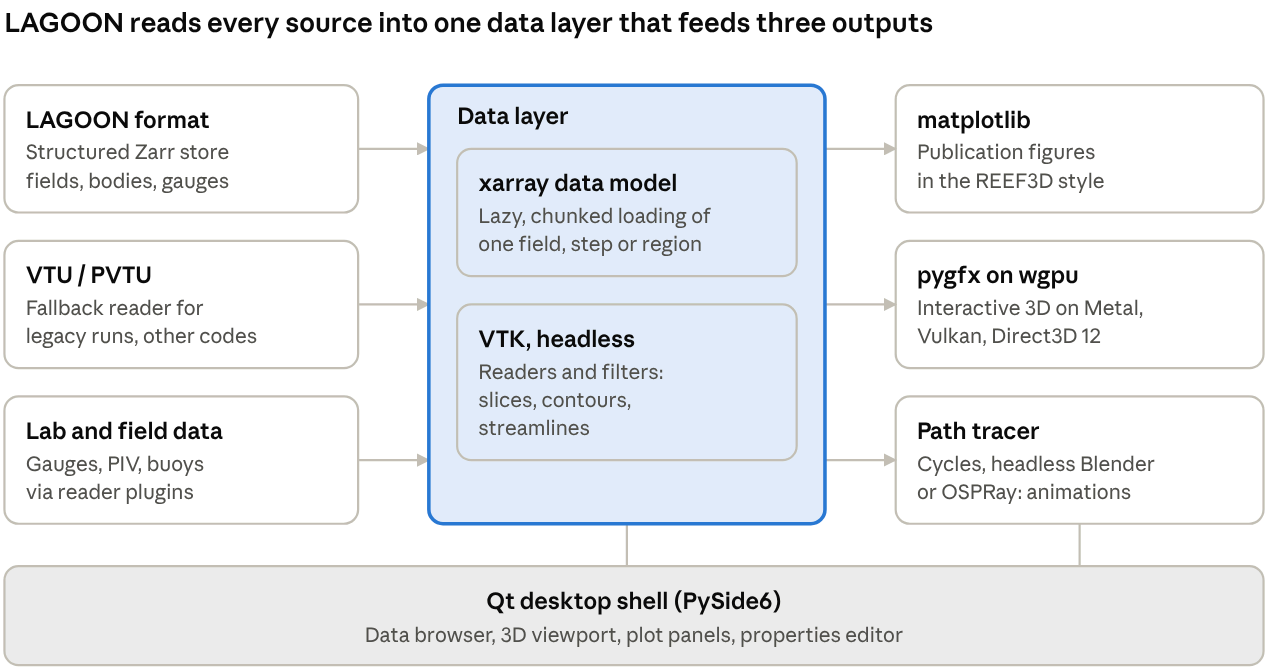
Design principles

Every overloaded program started out lean, so LAGOON's principles are meant to keep it that way.

1. **Task-first, not filter-first.** The interface speaks the language of the domain: free surface, gauges, forces, slices, wave fields, floating bodies. Generic filters exist but stay out of sight unless asked for.
2. **Opinionated defaults.** A dataset should look good with zero settings. Each new option is a maintenance cost, so options are added sparingly.
3. **Small core, plugins for the rest.** Readers, domain tools and specialised filters live outside the core, so the core can change without breaking users.
4. **Figure-first.** The exported figure or animation is the end product, and the whole workflow leads to it.
5. **Freedom to evolve.** Until version 1.0, interfaces may change. This is stated openly, so the project can move quickly in the years that shape it.

Architecture

LAGOON reuses proven libraries for the hard numerical work and replaces only the parts that make ParaView heavy: the user interface, the rendering and the figure output.



LAGOON architecture · 3 sources, 1 data layer, 3 outputs

All sources are read into one data layer; each output draws from it, so a figure, a 3D view and an animation of the same run always show the same data.

- **Data layer:** xarray is the in-memory model. Chunked storage means LAGOON loads only the field, time step or subregion being viewed, so large runs open quickly on a laptop.
- **VTK as a library, not an application:** VTK runs headless for what it does best, which is reading legacy formats and computing slices, contours, iso-surfaces and streamlines. Its interface, rendering and design culture are left behind.
- **Interactive 3D with pygfx:** pygfx renders through wgpu, which maps to Metal on macOS, Vulkan on Linux and Direct3D 12 on Windows. The same code gets native GPU performance on a Mac, a Linux cluster node and a Windows laptop.
- **Publication output with matplotlib:** line plots, spectra and 2D slices are drawn in matplotlib with house styles. 3D views are exported by compositing the GPU render with matplotlib axes, colorbars, labels and LaTeX equations, so 3D figures match the typography of the rest of a paper.
- **Desktop shell:** a Qt (PySide6) application hosts a data browser, the 3D viewport, 2D plot panels and a properties editor. The DataGraph-style editing, such as dragging variables onto axes and reusable figure templates, lives here.

The LAGOON data format

LAGOON defines a structured, chunked dataset layout on top of Zarr. It replaces the thousands of VTU/PVTU files of a run with one dataset, and stores only what a structured grid needs.

Store the grid, not the cells. For REEF3D's structured grids the geometry reduces to:

- 1D coordinate arrays x, y and σ (or z for Cartesian CFD grids)
- the bathymetry d(x, y)
- the free-surface elevation η(x, y, t)

The 3D σ-grid geometry is reconstructed on the fly and never written to disk:

```
z(x, y, \sigma, t) = \sigma \left( \eta(x, y, t) + d(x, y) \right) - d(x, y)
```

This alone should make output several times smaller than VTU; the exact factor is to be measured on reference cases.

Why Zarr. Zarr stores each chunk as a separate binary object with a small JSON description. This maps directly onto REEF3D's MPI decomposition: every rank writes its own chunks, with no locking and no parallel-I/O library. The layout is simple enough for REEF3D to write from C++ without any dependency, and xarray reads it natively in Python.

Container	Parallel writing from REEF3D	Dependency for REEF3D	Python access
Zarr (chosen)	One chunk per rank, no locking	None required	Native in xarray
HDF5	Needs an MPI-IO build and collective operations	HDF5 library	h5py, xarray
ADIOS2	Best performance at exascale	Large C++ library	adios2 bindings

What a dataset holds:

- **Fields:** 2D fields such as η for NHFLOW and FNPF, 3D fields such as velocity, pressure and the level set φ for CFD, chunked by space and time.
- **Bodies:** floating bodies and immersed boundaries stored once as a triangle mesh, plus one position and rotation per time step from the 6DOF solver.

- **Gauges and forces:** time series in the same dataset, so fields and probes are always in sync.
- **Metadata:** the case settings from ctrl.txt, solver version and units.

Open by design. The layout follows the xarray data model and CF-style names and units, and it will be documented openly. A small ParaView reader plugin gives users of other tools access, so data is never locked into LAGOON.

VTU as a fallback. VTU and PVTU files are still read through meshio or headless VTK, including stitching MPI partitions together. Old runs and other codes remain usable.

The format has value on its own: REEF3D gains smaller output, faster I/O and direct Python analysis even before the LAGOON application exists.

Cinematic rendering without Blender's complexity

LAGOON offers the small part of Blender that CFD results need, which is realistic materials, lighting and camera animation, and hides it behind presets. Modeling, rigging, UV mapping and node editors stay out.

Two rendering modes, one scene. The same scene description drives both, so the fast preview closely matches the final render.

- **Real-time mode:** pygfx's physically based materials and environment maps give water with Fresnel reflection, transparency and depth-based colour at interactive frame rates on Metal. This game-engine quality is enough for most talks.
- **Path-traced mode:** for showcase animations, LAGOON hands the scene to a renderer used as a library, not as an application:
  - **Headless Blender:** LAGOON writes the scene and runs Blender in the background as a separate process. The user clicks Render and never sees Blender. Running it as a separate process keeps LAGOON's own license independent of Blender's GPL.
  - **Cycles embedded directly:** Blender's renderer is Apache-licensed and supports Metal on Apple Silicon, but embedding it without Blender takes more integration work.
  - **OSPRay:** Intel's ray tracer, already used by ParaView and VisIt for scientific data.

Presets instead of settings:

- **Scene templates:** open ocean at sunset, wave flume with glass walls, underwater view, aerial coastline.
- **Materials by meaning:** water, sand bed, rock, concrete, steel, with no shader nodes.

- **Automatic mapping from REEF3D data:** η or the level set becomes the water surface, the sediment bed becomes sand, foam comes from the breaking indicator or air fraction, and DEM or MP-PIC particles become instanced grains. Floating bodies move with their 6DOF data.
- **Camera paths:** a few views clicked in the viewer become keyframes on a smooth path. Options such as follow the wave crest or orbit the structure are single choices.

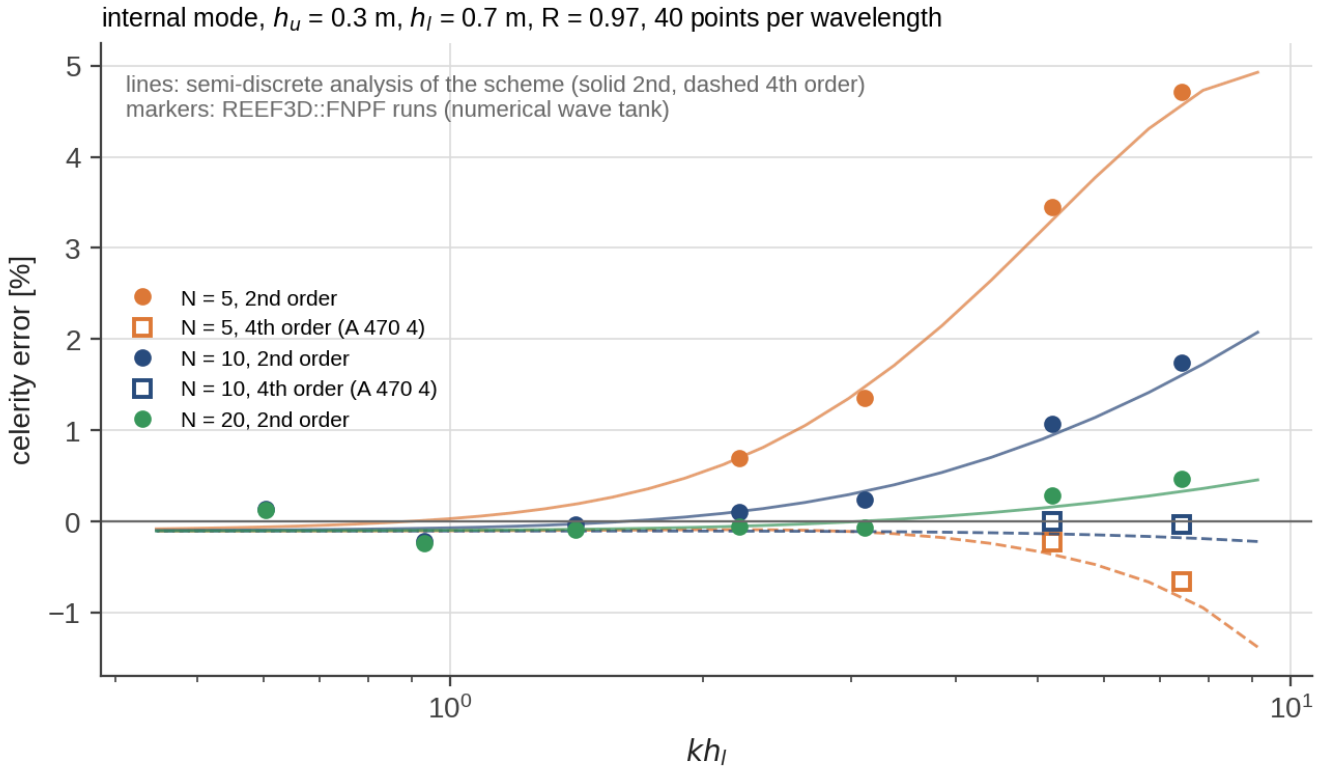
The data format makes this practical. Because body geometry is stored once and only its motion per time step, a moving body or evolving water surface is cheap to pass to a renderer, whereas importing thousands of mesh files into Blender is slow.

Example: recent REEF3D studies

Three REEF3D studies from September 2026 show today's post-processing workflow: every study carries its own hand-written plotting and animation scripts, about 1,660 lines in total, none of them reusable. LAGOON would turn this work into shared templates.

Study	Solver	Figures	Animations	Analysis scripts
Two-layer internal waves	FNPF	12	4	10 scripts, 388 lines
Ice floes: crushing and fracture	FNPF with ice module	8	8	1 script, 282 lines
Rubble-mound breakwater with DEM armour	NHFLOW with DEM	9	16	11 scripts, 992 lines

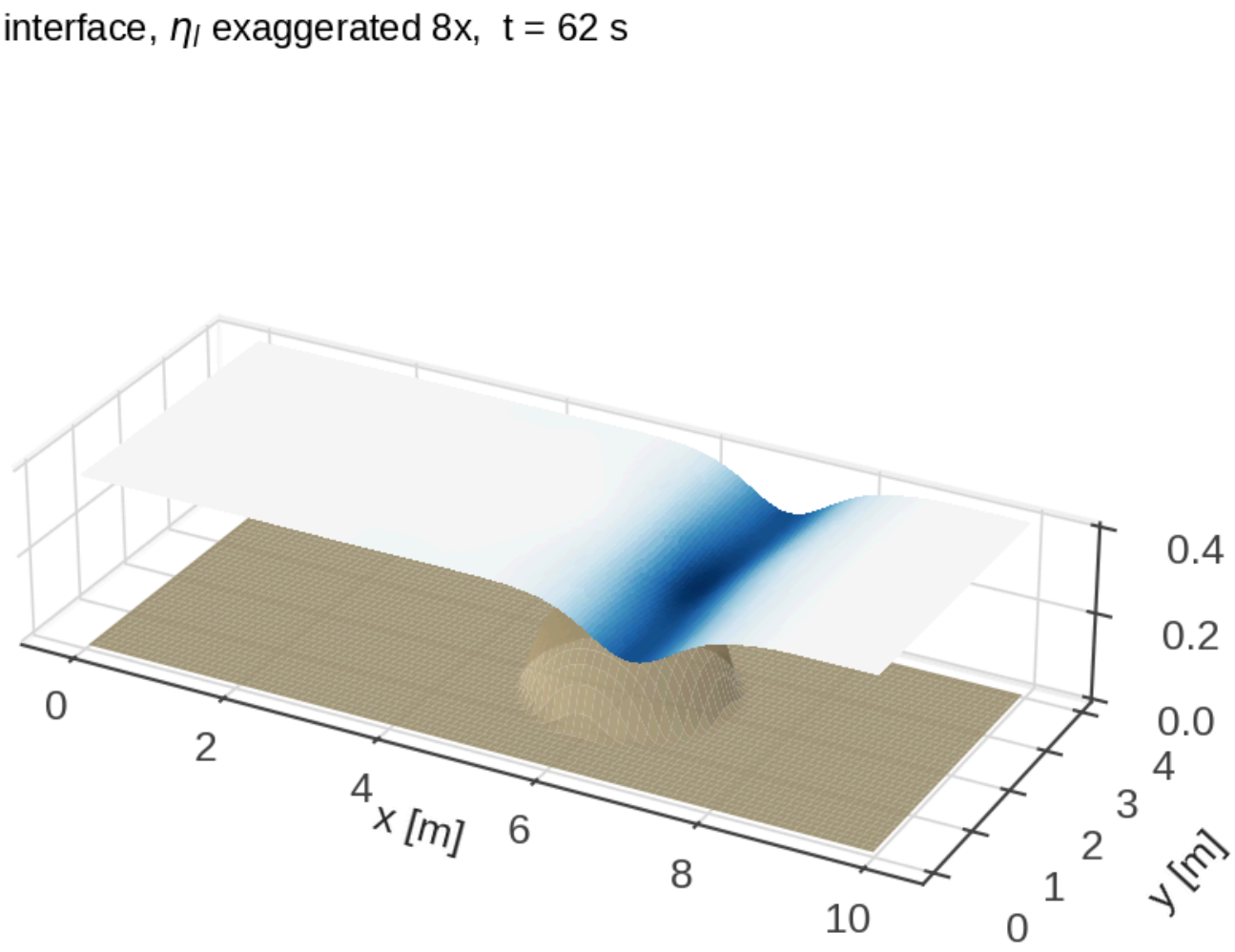
Publication figure: internal-wave dispersion



The celerity error of REEF3D::FNPF runs (markers) is compared with the semi-discrete analysis of the scheme (lines). Today, dedicated scripts read the gauge output of each run, compute the wavenumber and draw the figure.

In LAGOON, this becomes a validation template: gauges come straight from each run's dataset, theory curves are an overlay layer, and the house style is applied automatically. Re-running a case, for example with more σ-levels, updates the figure without touching a script.

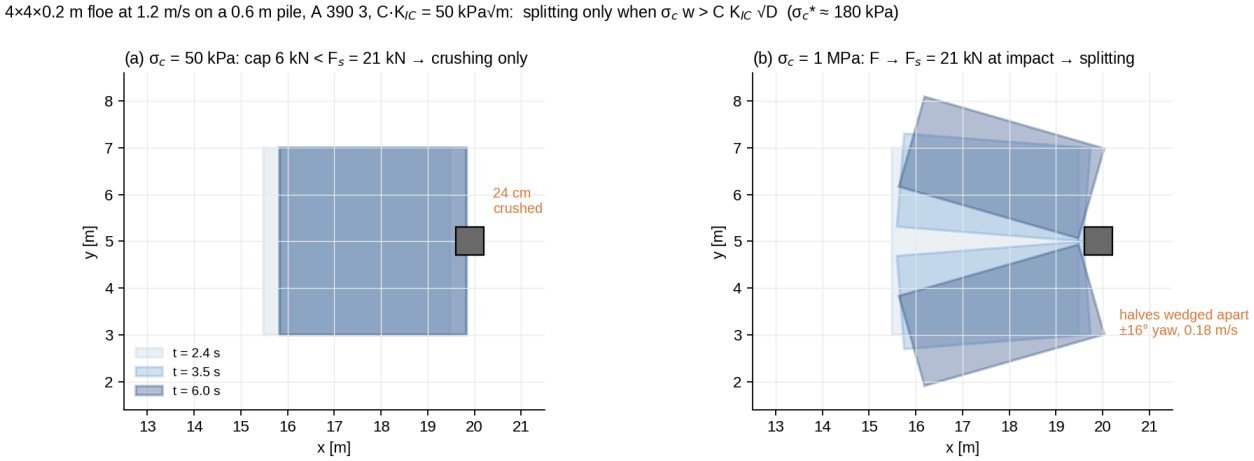
3D view: internal solitary wave over a seamount



This perspective view is drawn with matplotlib's 3D module: a camera angle fixed in the script, one time step per render, and the exaggeration passed as a command-line argument. Each new view means editing and re-running the script.

In LAGOON, the interface η_I and the bed are GPU surfaces rendered on Metal. The user rotates the view, scrubs through time and sets the vertical exaggeration with a slider, then exports the same view as a figure or an MP4. In the data format, both layers of the two-layer σ-grid are structured arrays and the interface is a 2D field, so the 3D geometry is rebuilt on the fly.

Bodies and contact: an ice floe hitting a pile



Floe outlines at several time steps are overlaid to show crushing against splitting. Today a study-specific script parses REEF3D's VTP ice output and floe logs itself, and merges the MPI pieces of the free-surface files by hand.

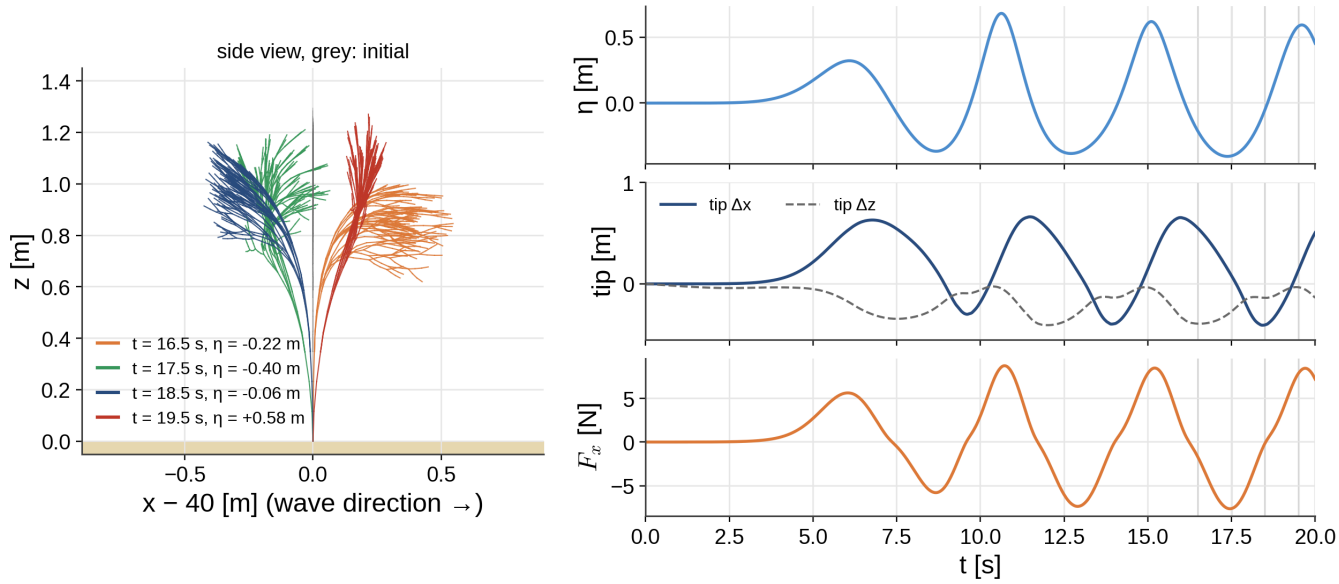
In LAGOON, floes are bodies: a mesh stored once plus a pose per time step, with fractured pieces added as new bodies. Time-step overlays are a built-in option. The breakwater study fits the same body layer: its 144 armour stones become instanced meshes, coloured by displacement, which the viewer can render interactively in 3D.

These three studies would make good first test cases and gallery entries for the first public releases.

Gallery: current REEF3D output

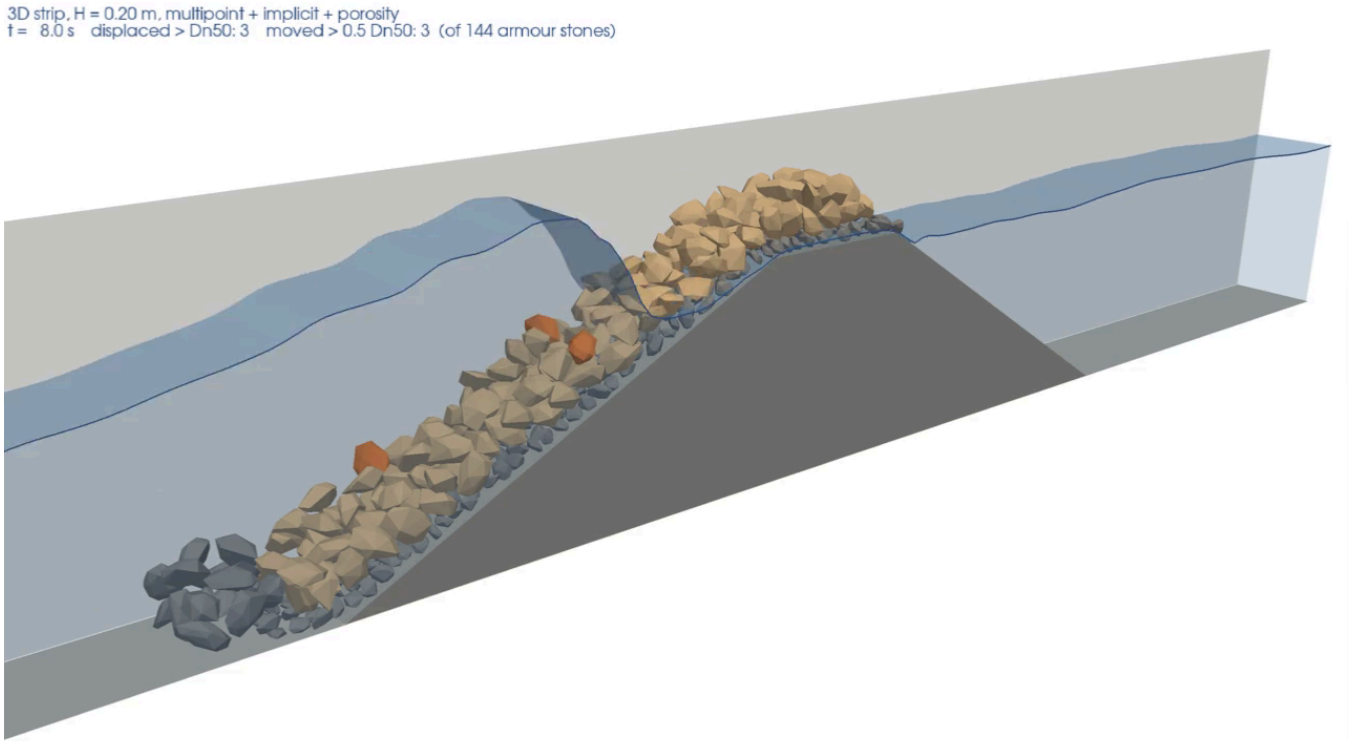
These screenshots from the September 2026 hans_dev work show the range of data LAGOON has to handle: flexible structures, thousands of contacting particles, floating bodies and 2D analysis fields. Each was produced with scripts written for that study.

RodTree: soft-coral fan colony in Stokes waves (NHFLOW)



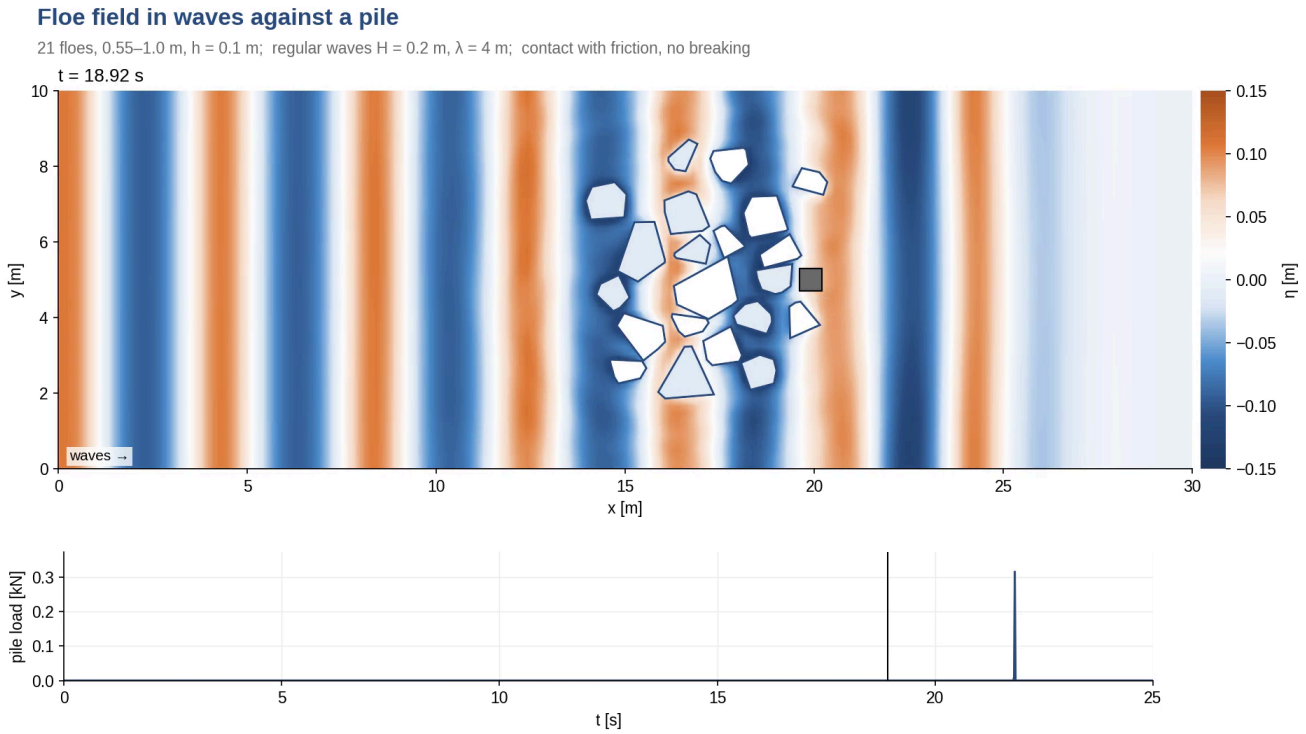
A 742-node Cosserat rod tree, 1.29 m high, in waves with H = 1.0 m and T = 4.5 s. In LAGOON, the tree is a line-mesh body with a pose per node and time step, drawn in 3D next to the linked time series of tip displacement and base force.

DEM breakwater: armour stones under wave attack (NHFLOW + DEM)



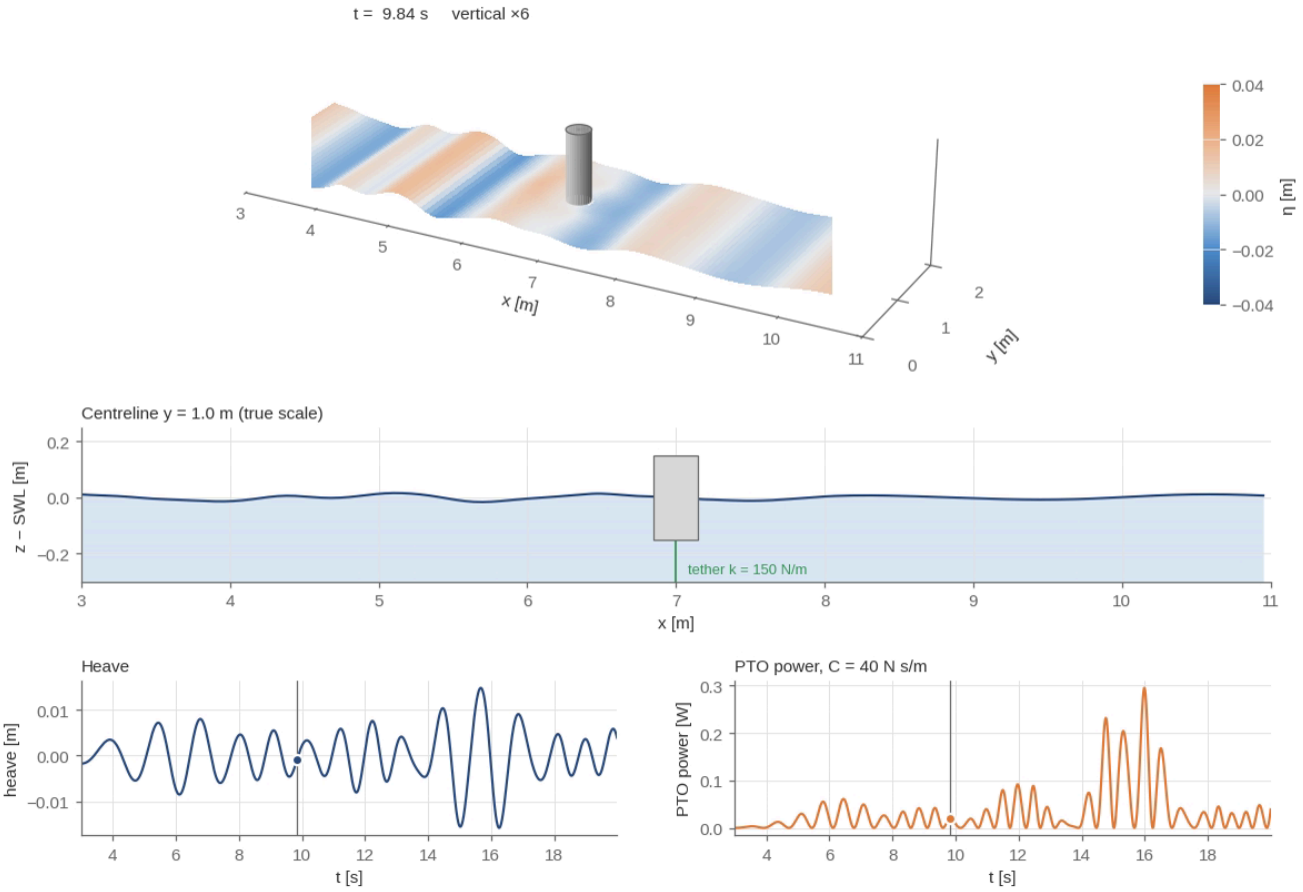
A 3D flume strip with 144 irregular quarry stones on a 1:2 slope. In LAGOON, each stone shape is stored once and instanced per time step, so the same scene can be rotated interactively and rendered with realistic rock and water materials.

DEM ice: floe field against a pile (FNPF)



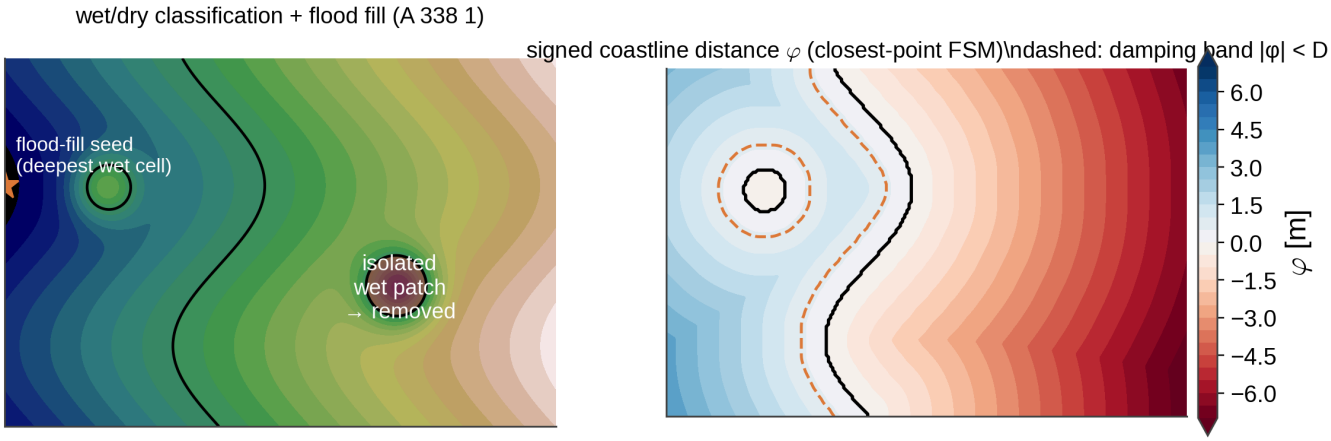
A field of rigid floes with contact and friction, drifting in waves onto a pile. In LAGOON, floes are bodies on top of the FNPF free surface, and the pile load is a linked time series whose cursor follows the animation.

FNPF WEC: heaving point absorber in JONSWAP waves



A resolved floating buoy with a PTO and tether in irregular waves (Hs = 5 cm). This single frame already combines a 3D view, a section and two time series, which is exactly the linked layout LAGOON is meant to provide without custom code.

FNPF coastline: fast sweeping and isolated wet areas



The wet/dry classification and the signed coastline distance on a synthetic bathymetry. The distance field comes from closest-point propagation with the fast sweeping method (fnpf_coastline_fsm.cpp), which replaces PDE reinitialisation. In LAGOON, such 2D analysis fields are layers over the bathymetry, with contour lines and colour maps from the house style.

DEM scour protection is not shown: no figures or results for it were found among the recent outputs.

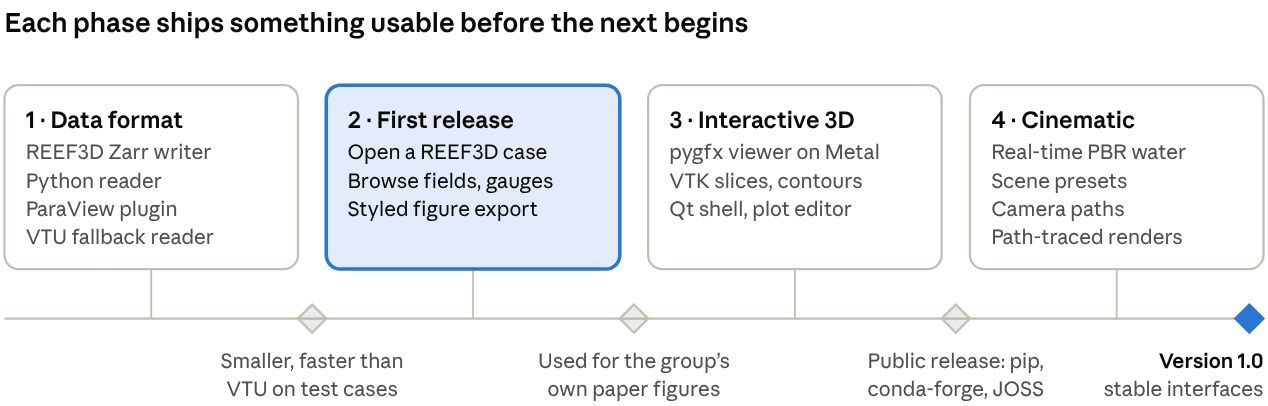
Open-source strategy

LAGOON is developed in the open under the REEF3D organisation on GitHub, with the aim of a community well beyond REEF3D users.

- **Beyond REEF3D:** REEF3D is the first and best-supported source. Readers for OpenFOAM, SWASH, XBeach, wave-tank and field data, and plain CSV come as plugins. A larger user base brings more contributors, and REEF3D benefits as the reference case.
- **License:** the stack is permissively licensed (VTK, PyVista, matplotlib and NumPy under BSD-style licenses; PySide6 under LGPL). A BSD-3-Clause or MIT license would match it and make adoption easy for institutes and companies. GPL would protect against closed forks but slows adoption and could complicate commercial add-ons by REEF Consulting. The license should be fixed before the first outside contribution.
- **Distribution:** installation with one command via pip or conda-forge on macOS, Linux and Windows.
- **Documentation and gallery:** example cases with data, and a gallery of figures and animations. In scientific software, a strong gallery often does more for uptake than a feature list.
- **Credit:** a paper in the Journal of Open Source Software (JOSS), so contributors and students receive citable credit.
- **Governance:** a small core team at NTNU at first, with NumFOCUS affiliation as a later step to show LAGOON is a community project rather than one lab's tool.

Roadmap

LAGOON is built in four phases, each ending in something useful on its own, starting with the data format.



LAGOON roadmap · 4 phases, gates between

Phase 2, the first release, is the key milestone: a small tool that opens a REEF3D case, browses fields and gauges, and exports a figure in the house style. People adopt tools that solve one problem immediately.

- **Phase 1, data format:** a Zarr writer in REEF3D, a Python reader, a ParaView reader plugin and the VTU fallback. It benefits REEF3D users directly.
- **Phase 2, first release (v0.1):** case browser, gauge and field plots, and styled figure export through matplotlib.
- **Phase 3, interactive 3D:** the pygfx viewer, VTK-based slices, contours and streamlines, and the Qt shell with DataGraph-style plot editing. Ends with the public release and a JOSS paper.
- **Phase 4, cinematic rendering:** real-time water materials, scene presets, camera paths and path-traced output, leading to version 1.0.

A working prototype of phases 1 and 2 is a matter of weeks. A polished tool that users choose over ParaView is a multi-year effort.

Resources, funding and risks

Open-source projects usually fail from a lack of maintainers, not a lack of ideas, so LAGOON needs dedicated people from the start.

People: at least one research software engineer, or a PhD project combining visualization and hydrodynamics, plus contributions from REEF3D developers for the format writer.

Funding routes:

- **A work package in a larger proposal:** OCEANiC is a natural fit, since a common post-processor for REEF3D::FNPF, REEF3D::CFD, OpenFOAM and ReFRESCO output matches its consolidation goal.
- **Research infrastructure or open-science funding** for the format and the public release.
- **REEF Consulting:** commercial add-ons and support services built on the open core, if the license allows it.

Risks:

Risk	Mitigation
Scope creep turns LAGOON into another overloaded tool	Design principles, small core, features as plugins
Too few maintainers after the first funding ends	Community beyond REEF3D, JOSS credit, NumFOCUS affiliation
pygfx and wgpu are young libraries	VTK as a fallback renderer for 3D
Users stay with ParaView out of habit	ParaView reader plugin for the format, strong gallery, one-command install
License conflict with GPL components	Blender only as a separate process; license fixed early

Name and identity

The software is called LAGOON, short for Lightweight Animation and Graphics for Open Ocean Numerics. "Open" stands for both the open ocean and open source.

- **Home:** a repository under the REEF3D organisation on GitHub, which ties LAGOON to the REEF3D family while the plugin design keeps it open to other codes.
- **Package names:** `lagoon` is already taken on PyPI by an unrelated package, so the Python package would be published as `lagoon-viz`, imported as `lagoonviz` to avoid a clash. `lagoon` is still free on conda-forge.
- **Search visibility:** amazee.io's Lagoon, an open-source application delivery platform, dominates web searches for the name. Writing "REEF3D LAGOON" in papers and on the web helps users find the project.

- **Trademark:** neither existing Lagoon is in scientific visualization, but a trademark check, for example through NTNU's technology transfer office, is advisable before a public launch.
- **Scope note:** "Numerics" suggests simulation output only, so the documentation should state clearly that lab and field data are supported too.

Sources

- lagoon on PyPI
- uselagoon/lagoon on GitHub (amazee.io)